

# dMoE: dLLMs with Learnable Block Experts

Sicheng Feng, Zigeng Chen, Gongfan Fang, Xinyin Ma, Xinchao Wang\*  
National University of Singapore  
fengsicheng@u.nus.edu, xinchao@nus.edu.sg

## Abstract

Diffusion Large Language Models (dLLMs) have recently emerged as a promising alternative to autoregressive models, offering competitive performance while naturally supporting parallel decoding. However, as dLLMs are increasingly integrated with Mixture-of-Experts (MoE) architectures to scale model capacity, a fundamental mismatch arises between block parallel decoding and token-level expert selection. Specifically, each dLLM forward pass processes multiple tokens with bidirectional dependencies, whereas conventional MoE layers route each token independently. This mismatch substantially increases the number of uniquely activated experts, making inference increasingly memory-bound. To address this, we propose dMoE, a simple yet effective block-level MoE framework. The central idea of dMoE is to aggregate token-level expert distributions within each block into a unified block-level expert distribution, which is then used to guide block expert routing in a coherent manner. In this way, dMoE substantially reduces the number of uniquely activated experts during inference without sacrificing performance, thereby mitigating the memory-bound bottleneck. Extensive experiments across a variety of benchmarks demonstrate the effectiveness of dMoE. On average, dMoE reduces the number of uniquely activated experts from 69.5 to 14.6 while retaining 99.11% of the original performance. Meanwhile, it reduces memory usage by 76.64% to 79.84% and achieves  $1.14\times$  to  $1.66\times$  end-to-end latency speedup. Code is available at: <https://github.com/fscdc/dMoE>.

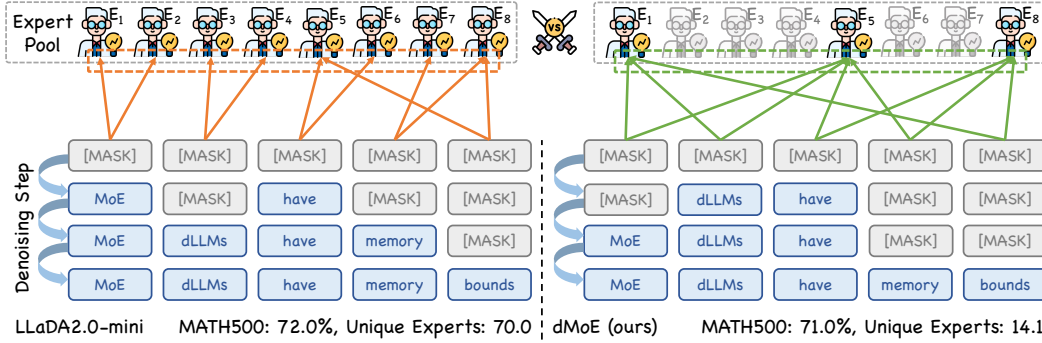


Figure 1: Comparison between the original LLaDA2.0-mini and our proposed dMoE. Unlike the original token-level expert routing in MoE dLLMs, our dMoE replaces token-level routing with block-level routing, substantially reducing the unique expert count while preserving performance.

## 1 Introduction

Recently, Diffusion large language models (dLLMs) [1, 2, 3, 4, 5] have emerged as a competitive alternative to autoregressive LLMs [6, 7, 8], demonstrating strong performance in both open-source

\*Corresponding Author

and closed-source settings [9, 10]. By iteratively refining mask tokens through an unmask-and-remask process, dLLMs naturally support parallel decoding beyond the left-to-right generation order of autoregressive models, offering greater flexibility and efficiency potential at test time. To further scale model capacity while keeping the number of active parameters manageable, recent dLLMs [11, 12, 13, 14, 15] have increasingly been integrated with Mixture-of-Experts (MoE) architectures, making MoE a growing design trend in this paradigm.

While MoE architectures offer an effective scaling strategy by increasing model capacity with sparse activation, they also introduce a fundamental efficiency challenge. In MoE dLLMs, existing MoE routing still follows the token-level expert selection paradigm inherited from autoregressive models, selecting experts independently for each token. However, dLLMs process multiple tokens within a single forward pass (e.g., a whole block of tokens in block diffusion decoding [16]). As a result, the number of uniquely activated experts can grow dramatically within one forward pass, making memory access the primary inference bottleneck. The empirical results in Section 3 further support our claims by showing that MoE latency dominates the end-to-end inference latency and linearly increases with the number of uniquely activated experts.

A large body of prior work has studied efficient MoE strategies, primarily in autoregressive models. Existing methods can be broadly divided into two categories: pre-execution compression, such as expert pruning [17, 18, 19, 20, 21] and expert merging [22, 23, 24], and runtime-adaptive execution, such as expert skipping [25, 26, 27], adaptive expert selection [28], and expert reuse [29, 30]. However, MoE efficiency in dLLMs remains largely underexplored. Unlike autoregressive decoding, dLLMs perform parallel token generation and refinement within each denoising step, making expert activation patterns and efficiency bottlenecks fundamentally different. A few recent studies have begun to explore this setting. EC-DLM [31] replaces token-choice routing with expert-choice routing and improves load balancing by dynamically adjusting expert capacity. TEAM [32] leverages temporal and spatial consistency in routing to reuse experts across denoising steps, thereby reducing overall expert activation during inference. DES [33] explicitly targets the memory overhead caused by excessive expert activation through a candidate-constrained routing strategy.

We propose dMoE, a simple yet effective strategy for compressing unique experts in MoE dLLMs. Our design is motivated by two key observations. First, token-level expert scores provide an informative signal of expert importance. Second, the degree of expert concentration varies substantially across denoising steps and blocks. Specifically, we first aggregate token-level expert scores to form block-level expert scores, and then use these block-level scores to guide the original routing process, thereby controlling the number of uniquely activated experts. In this way, dMoE can aggressively reduce the unique expert count without changing the number of experts selected for each token. Moreover, dMoE dynamically controls the unique expert count with a top- $p$  criterion, allowing it to better adapt to the varying routing characteristics across different denoising steps and blocks. During training, we adopt a self-distillation paradigm using the same routing procedure in the forward pass.

We choose LLaDA2.0-mini, a state-of-the-art open-source dLLM, as the base model for fine-tuning and evaluation. We evaluate dMoE on four benchmarks, including MATH500 [34], GSM8K [35], ARC-C [36], and MMLU [37]. The results show that dMoE consistently achieves substantial expert compression with no performance degradation (as shown in Figure 1). On average, dMoE reduces the number of uniquely activated experts by  $4.77\times$  while retaining 99.11% of the original performance. In addition, it reduces memory usage by 76.64% to 79.84% and delivers  $1.14\times$  to  $1.66\times$  end-to-end latency speedup compared with the original model. Our dMoE also achieves a superior performance-efficiency trade-off compared with the baselines. Furthermore, our dMoE is tunable, allowing the number of activated experts to be adjusted to different application requirements.

Overall, we introduce dMoE, a novel learnable strategy for block-level expert routing in MoE dLLMs. The core idea of dMoE is to aggregate token-level expert scores into block-level expert scores, and then use these block-level scores to dynamically guide the original routing process. Extensive experiments demonstrate the effectiveness of our method. This work establishes a strong baseline for block-level routing in MoE dLLMs.

## 2 Related Work

**Overview of Diffusion Language Models.** Diffusion-based generative modeling has achieved remarkable success in continuous modalities, including images [38, 39], videos [40, 41], and au-

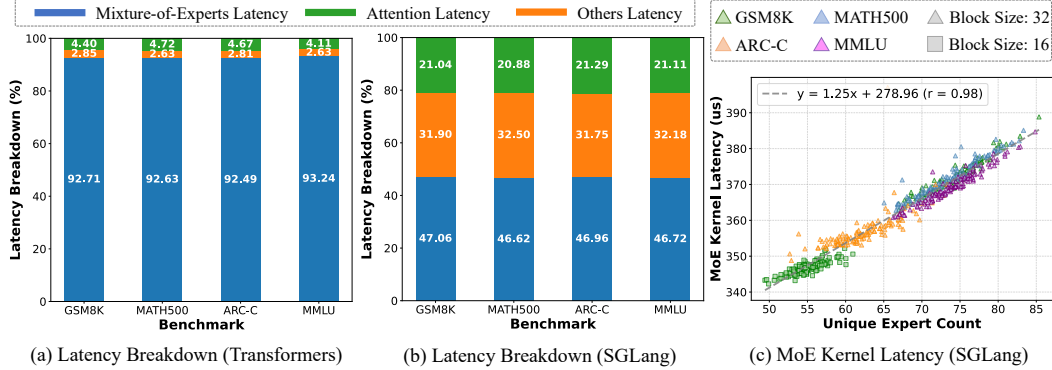


Figure 2: Empirical studies on LLaDA2.0-mini. (a) & (b) We report the latency breakdown of three components: MoE (*e.g.*, routing and execution), attention (*e.g.*, attention computation and related projections), and others (*e.g.*, LM head and embeddings). (c) We present the relationship between the number of unique activated experts and MoE kernel latency.

dio [42, 43], building on the broader foundation of diffusion models [44, 45, 46]. Extending this framework to language, however, is non-trivial because text is inherently discrete. To address this challenge, a growing body of work formulates diffusion directly in token space [47, 48, 49, 50, 14, 3], often through masked-token denoising, which enables parallel generation and relaxes the strict left-to-right dependency of autoregressive decoding. Based on this formulation, dLLMs [3, 4, 10, 9, 11] have shown increasingly competitive performance at the billion-parameter scale, suggesting that diffusion is becoming a practical alternative for language generation. Recent progress also suggests a clear trend toward sparse scaling in dLLMs, with an increasing number of representative models adopting MoE architectures to expand overall model capacity while keeping the number of active parameters at each denoising step relatively small [13, 11, 12]. Beyond general text generation, diffusion-based language modeling is now being extended to more challenging settings, including reasoning [51, 52, 53, 54, 55, 56], multimodal generation [57, 58, 5, 59], and code synthesis [60, 10, 61], highlighting the rapid expansion and growing maturity of this research direction [62, 63, 64].

**Efficient Mixture-of-Expert Strategies.** In autoregressive models, most existing studies in this setting are designed to reduce token-wise expert computation or serving overhead, and are therefore primarily tailored to left-to-right decoding. Broadly, these methods can be grouped into two lines. The first line focuses on pre-execution compression, which reduces the model-side budget before inference, for example, through expert pruning [17, 18, 19, 20, 21] and expert merging [22, 23, 24]. The second line focuses on runtime-adaptive execution, which improves efficiency during inference by dynamically controlling expert activation according to the current input, such as expert skipping [25, 26, 27], adaptive expert selection [28], and expert reuse [29, 30]. Beyond these algorithmic strategies, another important direction lies in system-level optimization [65, 66], which improves MoE efficiency from the perspectives of communication and memory.

However, MoE efficiency in dLLMs remains largely underexplored. Some recent studies have provided initial evidence showing its potential. EC-DLM [31] replaces conventional token-choice routing with expert-choice routing and further improves load balancing by dynamically adjusting expert capacity across denoising steps. TEAM [32] exploits temporal and spatial consistency in expert routing to reuse experts across denoising steps, thereby reducing the overall number of activated experts during inference. DES [33] explicitly targets the memory overhead caused by excessive expert activation within each block under parallel decoding. It introduces a candidate-constrained routing strategy to compress the number of block-level activated experts without any additional training. Our target is closely aligned with DES, but pushes this direction further by pursuing more aggressive block-level expert selection to achieve stronger expert compression.

### 3 Preliminaries

We briefly introduce dLLMs from the perspective most relevant to this work: their native support for parallel decoding. Unlike autoregressive language models, which generate tokens strictly from left to right, dLLMs predict multiple unknown positions simultaneously by iteratively refining a partially

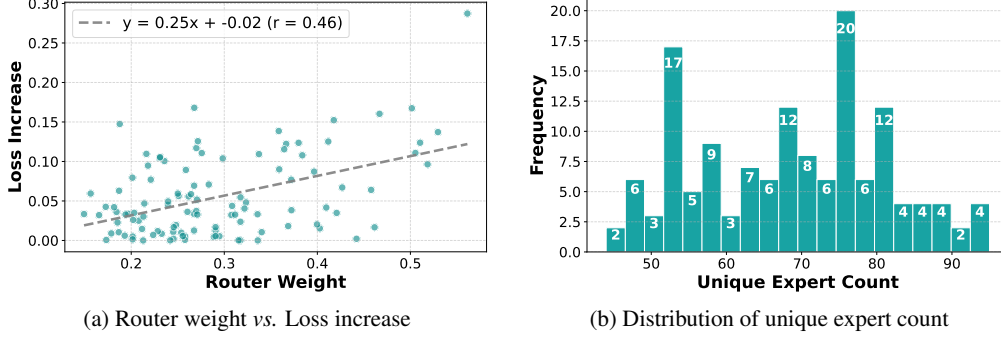


Figure 3: (a) We demonstrate the correlation between the router weights (token-level expert scores) and loss increase, drawn from GSM8K. (b) We present the distribution of the unique expert count during the inference process, drawn from GSM8K.

masked sequence. This property makes them especially suitable for parallel generation, since the model can update a set of masked tokens in one denoising step rather than committing to a single next token at a time. A common instantiation is the masked diffusion language model (MDLM), where generation starts from a corrupted sequence  $x_t$  obtained by masking tokens from a clean sequence  $x_0$ . Let  $t \in [0, 1]$  denote the masking intensity. The forward corruption process independently replaces each token with [MASK] according to

$$q(x_t | x_0) = \prod_{i=1}^L [(1-t) \delta(x_t^i = x_0^i) + t \delta(x_t^i = [\text{MASK}])]. \quad (1)$$

Given the corrupted sequence, the denoising model  $p_\theta$  predicts the original tokens at the masked positions:

$$p_\theta(x_0 | x_t) = \prod_{i: x_t^i = [\text{MASK}]} p_\theta(x_0^i | x_t). \quad (2)$$

Because all masked positions can be reconstructed in parallel within each denoising step, dLLMs naturally enable non-autoregressive parallel decoding.

**Memory Bound Becomes the Primary Bottleneck for Mixture-of-Experts dLLMs.** In MoE dLLMs, each forward pass needs to select experts for multiple tokens simultaneously (e.g., a full block of tokens under block diffusion decoding [16]). As a result, a large number of unique experts can be activated within a single forward pass, and these experts need to be iteratively loaded during inference, leading to substantial memory overhead. We further conduct empirical studies on LLaDA2.0-mini across various benchmarks to support this claim. Specifically, for all empirical studies, we use block diffusion decoding and include a warm-up stage before measurement to stabilize memory, cache, and kernel states. As shown in Figure 2 (a) and (b), we report the latency breakdown of three components: MoE, attention, and others. Here, MoE includes routing and expert execution, attention includes attention computation and related projections, and others include components such as the LM head and embeddings. We report results under both the native Transformers<sup>2</sup> and SGLang<sup>3</sup> framework, and consistently observe that MoE latency dominates the end-to-end latency. In Figure 2 (c), we additionally record the MoE kernel latency together with the corresponding unique expert count, i.e., the number of uniquely activated experts within a single forward pass; under block diffusion, this corresponds to the unique experts activated by all tokens in the current block. The results show a clear linear positive correlation, indicating that activating more unique experts leads to higher MoE kernel latency. Overall, these findings motivate us to alleviate the memory bottleneck in MoE dLLMs by reducing the number of unique activated experts.

## 4 Methods

Based on the analysis in Section 3, we focus on reducing the number of block-level unique experts to alleviate the memory bottleneck while keeping the computation unchanged, that is, maintaining a

<sup>2</sup><https://github.com/huggingface/transformers>

<sup>3</sup><https://github.com/sgl-project/sglang>

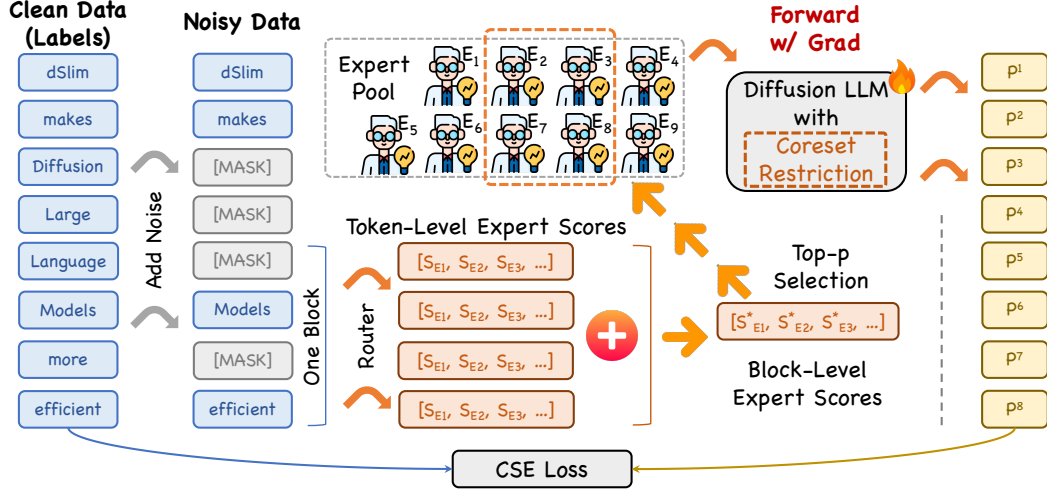


Figure 4: Overview of our proposed dMoE. For each noisy block, we aggregate token-level router scores into block-level expert scores and apply a top-p criterion to select an adaptive expert coreset. The final token-level routing is then restricted to this coreset. Training follows a self-distillation paradigm using CSE loss, and inference uses the same forward process.

fixed number of selected experts for each token. In this section, we first present two key observations in Section 4.1, and then introduce our proposed dMoE in detail in Section 4.2.

#### 4.1 Key Observations

We conduct empirical studies of block diffusion decoding on LLaDA2.0-mini using GSM8K. From these results, we identify several key observations that motivate the design of our dMoE for expert compression in MoE dLLMs.

**Observation A.** *Token-level expert scores provide an effective indicator of expert importance.* To verify this, we analyze the relationship between token-level expert scores (i.e., router weights) and the change in reconstruction loss after masking the corresponding experts. Specifically, we randomly sample 108 data points from GSM8K, using the original LLaDA2.0-mini as the base model, and compute the correlation between these two quantities. As shown in Figure 3 (a), they exhibit a clear positive correlation, with a Pearson correlation coefficient of 0.462. This observation motivates us to aggregate token-level expert scores for block-level expert selection.

**Observation B.** *The degree of expert concentration varies substantially across denoising steps and blocks.* To examine this phenomenon in more detail, we further analyze the distribution of the unique expert count for the generated block at Layer 10 during inference, where Layer 10 corresponds to the middle layer of LLaDA2.0-mini, which contains 19 layers in total. As illustrated in Figure 3 (b), the unique expert count changes noticeably across different denoising steps, suggesting that the extent to which tokens are routed to a concentrated subset of experts is highly step-dependent. To further quantify this variability, we adopt the coefficient of variation (CV), defined as

$$CV = \frac{\sigma}{\mu},$$

where  $\sigma$  and  $\mu$  denote the standard deviation and mean of the unique expert count, respectively. As shown in Table 1, the unique expert count exhibits consistently large variation across various benchmarks, suggesting that this phenomenon is not tied to a specific dataset or task. These results provide further empirical evidence that expert concentration varies substantially across denoising steps and blocks.

## 4.2 dMoE

To achieve our goal of reducing the number of block-level unique experts, a natural idea is to constrain the expert pool during expert selection. Following a coarse-to-fine strategy, we first select a coreset from the full expert pool, and then perform token-level expert selection within this coreset (Figure 4). Specifically, given the tokens within a block, we first perform token-level routing to compute token-level expert scores:

$$\mathbf{s}_i = \text{Router}(t_i) = [s_{E_1}, s_{E_2}, \dots, s_{E_{|\mathcal{E}|}}],$$

where  $t_i$  denotes the representation of the  $i$ -th token in the block, and  $s_{E_i}$  is the routing score of expert  $E_i$  for token  $i$ . Inspired by observation A, which suggests that token-level expert scores are positively correlated with expert importance, we then directly aggregate the token-level expert scores to obtain block-level expert scores:

$$S_{block} = \oplus_{i \in \mathcal{B}} \mathbf{s}_i,$$

where  $\mathcal{B}$  denotes the set of tokens in the current block, and  $\oplus$  represents the aggregation operator. After obtaining the block-level expert scores, we further determine the coreset based on their normalized values. Inspired by observation B, which suggests that the degree of expert concentration varies substantially across denoising steps and blocks, we do not enforce a fixed coreset size. Instead, we first normalize the block-level expert scores and then apply a top-p criterion to select the coreset:

$$\mathcal{C} = \text{Top-P}(\{\tilde{S}_{block}^E\}_{E=1}^E, p),$$

where  $\mathcal{C}$  denotes the selected coreset and  $p$  is the cumulative probability threshold. This design naturally yields a smaller coreset when the expert score distribution is concentrated, and a larger one when the distribution is more dispersed, thereby adapting to the routing characteristics of different blocks. As such, it is better aligned with the original inference behavior. Finally, the original token-level routing is performed within the selected coreset (see more details in Algorithm 1).

During the training process, we adopt a self-distillation paradigm [67] and follow the above routing procedure in the forward pass. During the inference process, we use the same forward process to maintain alignment between training and inference.

## 5 Experiments

### 5.1 Experimental Setups

**Training Data.** All training data are constructed via self-distillation [67]. We first collect prompts from several public datasets, including the GSM8K training set [35], PRM12K [34], a subset of Numina-Math [68], and a subset of OpenThoughts [69]. We then use LLaDA2.0-mini (*i.e.*, our base model) to generate corresponding responses as supervision targets. During generation, we follow the official settings: the confidence threshold is set to 0.95, the block size to 32, and the maximum output length to 2048 tokens. Samples that fail to terminate within this length limit are removed. Importantly, the entire supervision signal is derived from the model’s own generations, without relying on any externally curated high-quality responses. Overall, we obtain approximately 700K training samples.

**Training Details.** We use LLaDA2.0-mini as the base model, which is a state-of-the-art open-source MoE dLLM. During training, we randomly sample the masking ratio from  $[0.3, 0.8]$  for each training instance. We perform full fine-tuning for 2 epochs with a global batch size of 4. The learning rate is set to  $2.0 \times 10^{-6}$  with a cosine learning rate schedule. We adopt the block diffusion setting and follow the official configuration by setting the block size to 32. The top-p threshold is set to 0.6 during training. All training is conducted on 4 H100 GPUs.

**Baselines.** We compare dMoE against four baselines: (1) **Original**, which follows the official configuration with block diffusion inference; (2) **Top-4**, which reduces the number of selected experts

---

#### Algorithm 1: Block-Level Expert Routing

---

**Input:** Token-level expert scores  $\{\mathbf{s}_i\}_{i \in \mathcal{B}}$

**Output:** Block-level expert scores  $S_{block}$ , coreset  $\mathcal{C}$ ,  
routed experts  $\{R_i\}_{i \in \mathcal{B}}$

Initialize  $S_{block} \leftarrow \mathbf{0} \in \mathbb{R}^{|\mathcal{E}|}$ ;

**foreach**  $i \in \mathcal{B}$  **do**

$\hat{\mathbf{s}}_i \leftarrow \text{TopKMask}(\mathbf{s}_i, k)$ ;  
     $S_{block} \leftarrow S_{block} + \hat{\mathbf{s}}_i$ ;

$\tilde{S}_{block} \leftarrow \text{Normalize}(S_{block})$ ;

$\mathcal{C} \leftarrow \text{TopP}(\tilde{S}_{block}, p)$ ;

**foreach**  $i \in \mathcal{B}$  **do**

$\mathbf{s}_i^{\mathcal{C}} \leftarrow \text{MaskOutside}(\mathbf{s}_i, \mathcal{C})$ ;  
     $R_i \leftarrow \text{TopK}(\mathbf{s}_i^{\mathcal{C}}, k)$ ;

**return**  $S_{block}, \mathcal{C}, \{R_i\}_{i \in \mathcal{B}}$ ;

---

Table 2: Results on various benchmarks with LLaDA2.0-Mini. For MMLU, we use the math part for evaluation. Since the official DES code is not publicly available, we implement DES for comparison. Expert count denotes the average number of unique activated experts per layer. \* indicates that the corresponding threshold in DES is adjusted to achieve a higher compression ratio.

| Method                     | MATH500      |          | GSM8K        |          | ARC-C        |          | MMLU         |          |
|----------------------------|--------------|----------|--------------|----------|--------------|----------|--------------|----------|
|                            | Expert Count | Accuracy | Expert Count | Accuracy | Expert Count | Accuracy | Expert Count | Accuracy |
| LLaDA2.0-Mini              | 70.0         | 72.0%    | 71.5         | 90.8%    | 65.9         | 84.4%    | 70.6         | 88.6%    |
| <i>Baselines</i>           |              |          |              |          |              |          |              |          |
| Top-4                      | 43.5         | 52.6%    | 44.3         | 72.4%    | 42.2         | 74.0%    | 43.8         | 61.5%    |
| DES-S                      | 41.0         | 70.8%    | 41.9         | 90.7%    | 39.9         | 83.3%    | 41.4         | 87.1%    |
| DES-V                      | 36.9         | 71.8%    | 37.0         | 90.5%    | 36.8         | 82.3%    | 37.0         | 85.6%    |
| DES-S*                     | 12.9         | 53.8%    | 13.2         | 74.0%    | 13.0         | 62.9%    | 13.0         | 70.7%    |
| DES-V*                     | 13.0         | 53.0%    | 12.9         | 73.5%    | 13.0         | 49.1%    | 13.0         | 72.6%    |
| <i>dMoE</i>                |              |          |              |          |              |          |              |          |
| ours ( $p^{train} = 0.6$ ) | 14.1         | 71.0%    | 14.7         | 88.6%    | 15.0         | 85.8%    | 14.5         | 87.4%    |
| ours ( $p^{train} = 0.5$ ) | 11.4         | 69.8%    | 11.6         | 87.1%    | 11.9         | 83.5%    | 11.6         | 87.0%    |

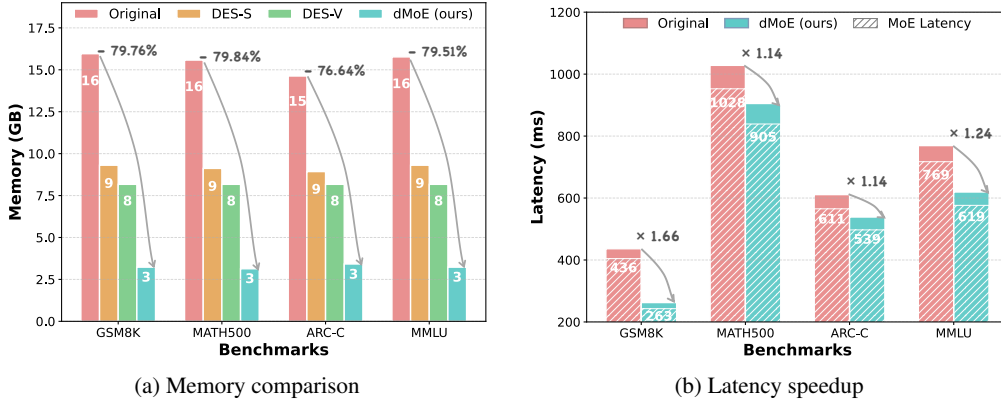


Figure 5: (a) We report the average memory footprint of uniquely activated MoE parameters across all layers for both the baselines and our dMoE. (b) We present the average end-to-end latency and MoE latency for the original model and our dMoE.

per token from 8 in the official setting to 4; (3) **DES-S**, where we implement DES-Seq [33], a sequence-level routing strategy that enables more adaptive expert allocation; and (4) **DES-V**, where we implement DES-Vote [33], which reduces the number of unique experts within each block by consolidating router-derived preferences across tokens. Additionally, for both DES-S and DES-V, the degree of expert compression can be controlled through hyperparameters.

**Inference Details.** We follow the official inference settings to evaluate the LLaDA2.0-mini [11]. For the baselines, we report the results from our implementation, with the latter following the configurations specified in the original paper. We apply the block diffusion setting with confidence-based parallel decoding, consistent with the official implementation. We follow the official settings for both our dMoE and all the baselines: set block size to 32, set max generation length to 2048, set confidence threshold to 0.95, and enable early stopping.

**Evaluation Details.** We conduct comprehensive experiments on multiple benchmarks spanning a broad spectrum of reasoning tasks, including mathematical reasoning, scientific reasoning, and general high-level reasoning. Specifically, we evaluate on MATH500 [34], GSM8K [35], ARC-C [36], and MMLU [37]. In addition, we follow the simple-eval framework<sup>4</sup> for zero-shot evaluation and prompt the model to generate its reasoning trajectories step by step.

## 5.2 Main Results

**dMoE Achieves Aggressive Expert Compression While Preserving Performance.** We report the main results in Table 2. Compared with the original model, our dMoE achieves substantial expert

<sup>4</sup><https://github.com/openai/simple-evals>

Table 3: Ablation on the cumulative probability threshold  $p$  at training and test stages. We evaluate cumulative probability thresholds of 0.4, 0.5, 0.6, 0.7, and 0.8 while keeping other settings fixed.

| Method                     | MATH500      |          | GSM8K        |          | ARC-C        |          | MMLU         |          |
|----------------------------|--------------|----------|--------------|----------|--------------|----------|--------------|----------|
|                            | Expert Count | Accuracy | Expert Count | Accuracy | Expert Count | Accuracy | Expert Count | Accuracy |
| LLaDA2.0-Mini              | 70.0         | 72.0%    | 71.5         | 90.8%    | 65.9         | 84.4%    | 70.6         | 88.6%    |
| dMoE ( $p^{train} = 0.6$ ) |              |          |              |          |              |          |              |          |
| ours ( $p^{test} = 0.4$ )  | 10.3         | 69.8%    | 10.3         | 85.0%    | 10.5         | 83.7%    | 10.3         | 85.9%    |
| ours ( $p^{test} = 0.5$ )  | 11.5         | 70.0%    | 11.7         | 87.0%    | 11.9         | 84.8%    | 11.6         | 87.0%    |
| ours ( $p^{test} = 0.6$ )  | 14.1         | 71.0%    | 14.7         | 88.6%    | 15.0         | 85.8%    | 14.5         | 87.4%    |
| ours ( $p^{test} = 0.7$ )  | 19.0         | 72.6%    | 19.9         | 90.5%    | 20.2         | 86.4%    | 19.6         | 87.8%    |
| ours ( $p^{test} = 0.8$ )  | 27.1         | 74.2%    | 28.1         | 91.7%    | 28.2         | 86.9%    | 27.9         | 89.3%    |
| dMoE ( $p^{train} = 0.5$ ) |              |          |              |          |              |          |              |          |
| ours ( $p^{test} = 0.4$ )  | 10.2         | 69.6%    | 10.3         | 86.0%    | 10.5         | 81.3%    | 10.3         | 84.4%    |
| ours ( $p^{test} = 0.5$ )  | 11.4         | 69.8%    | 11.6         | 87.1%    | 11.9         | 83.5%    | 11.6         | 87.0%    |
| ours ( $p^{test} = 0.6$ )  | 14.3         | 70.0%    | 14.7         | 88.3%    | 15.0         | 85.0%    | 14.5         | 87.0%    |
| ours ( $p^{test} = 0.7$ )  | 19.2         | 73.4%    | 20.1         | 90.1%    | 20.2         | 85.1%    | 19.7         | 88.5%    |
| ours ( $p^{test} = 0.8$ )  | 27.2         | 73.2%    | 28.0         | 90.8%    | 28.2         | 86.5%    | 28.0         | 88.9%    |

compression while preserving performance almost intact. Specifically, the average performance across four benchmarks decreases only slightly from 83.95% to 83.2%, retaining 99.11% of the original performance, while reducing the average unique expert count from 69.5 to 14.575, corresponding to a 79.04% reduction in the average unique expert count. Compared with the baselines, our dMoE further achieves a 60.47% to 66.44% greater reduction in unique expert count at a comparable performance level.

**dMoE Significantly Reduces Memory Usage.** As shown in Figure 5 (a), we compare the memory footprint of the original model, the baselines, and our dMoE. Specifically, we report the average memory footprint of uniquely activated MoE parameters across all layers. The results show that our dMoE achieves the lowest memory usage, yielding a 76.64% to 79.84% reduction compared with the original model and a 59.18% to 67.64% reduction compared with the baselines.

**dMoE Brings End-to-End Speedup.** We further report the end-to-end latency (including MoE latency), together with the corresponding speedup, in Figure 5 (b) to evaluate the practical efficiency of our method. Across four benchmarks, our method achieves  $1.14\times$  to  $1.66\times$  end-to-end latency speedup, with the gain mainly coming from the reduction in MoE latency. These results demonstrate the effectiveness of our method in delivering real inference acceleration.

**dMoE Performs Better at Extreme Expert Compression.** We further adjust the thresholds of DES-S and DES-V to achieve compression ratios comparable to those of our dMoE (DES-S\* and DES-V\* in Table 2). We can observe that under similar compression levels, our dMoE remains nearly lossless, retaining 99.11% of the original performance with  $p = 0.6$  and 97.50% with  $p = 0.5$ , whereas DES-S and DES-V suffer from substantial performance degradation. Notably, our dMoE can compress the number of unique experts to near the practical limit, *i.e.*, the number of experts required per token in the model, which is 8 in LLaDA2.0-mini, while still maintaining strong performance.

### 5.3 Diagnostic Results

We provide comprehensive ablation studies on the cumulative probability threshold  $p$  and block size. Furthermore, we show that our dMoE achieves a superior performance-efficiency trade-off compared with the baselines.

**Ablation on the Cumulative Probability Threshold  $p$ .** We first conduct an ablation study on the cumulative probability threshold  $p$ . Specifically, we train two models with  $p^{train} = 0.6$  and  $p^{train} = 0.5$ , respectively, and evaluate each tuned model under  $p^{test} \in \{0.4, 0.5, 0.6, 0.7, 0.8\}$ . As shown in Table 3, when evaluated with the same  $p^{test}$ , the two models trained with different  $p^{train}$  achieve similar performance, while still maintaining very low unique expert counts. Moreover, as  $p^{test}$  increases, the unique expert count gradually rises, accompanied by improved performance. Overall, these results demonstrate not only the robustness of our method but also its tunability, allowing the number of activated experts to be flexibly adjusted to different hardware characteristics and application requirements.

Table 4: Ablation on the block size. The evaluated model is trained with a block size of 32 with  $p^{train} = 0.6$ . We set  $p^{test} = 0.6$  here. We evaluate block sizes of 8, 16, 24, and 32 while keeping all other hyperparameters fixed.

| Method          | MATH500      |          | GSM8K        |          | ARC-C        |          | MMLU         |          |
|-----------------|--------------|----------|--------------|----------|--------------|----------|--------------|----------|
|                 | Expert Count | Accuracy | Expert Count | Accuracy | Expert Count | Accuracy | Expert Count | Accuracy |
| Block size = 32 |              |          |              |          |              |          |              |          |
| LLaDA2.0-Mini   | 70.0         | 72.0%    | 71.5         | 90.8%    | 65.9         | 84.4%    | 70.6         | 88.6%    |
| ours            | 14.1         | 71.0%    | 14.7         | 88.6%    | 15.0         | 85.8%    | 14.5         | 87.4%    |
| Block size = 24 |              |          |              |          |              |          |              |          |
| LLaDA2.0-Mini   | 60.7         | 73.6%    | 62.2         | 91.0%    | 57.8         | 82.3%    | 61.6         | 88.9%    |
| ours            | 15.4         | 74.4%    | 16.0         | 90.4%    | 16.4         | 87.2%    | 15.8         | 85.9%    |
| Block size = 16 |              |          |              |          |              |          |              |          |
| LLaDA2.0-Mini   | 48.6         | 71.2%    | 49.8         | 90.5%    | 47.0         | 86.9%    | 49.6         | 88.5%    |
| ours            | 15.9         | 71.2%    | 16.5         | 90.6%    | 16.8         | 85.6%    | 16.4         | 88.1%    |
| Block size = 8  |              |          |              |          |              |          |              |          |
| LLaDA2.0-Mini   | 31.5         | 71.0%    | 32.2         | 90.1%    | 31.1         | 87.7%    | 32.3         | 87.4%    |
| ours            | 14.2         | 73.0%    | 14.6         | 89.6%    | 15.1         | 86.5%    | 14.6         | 85.9%    |

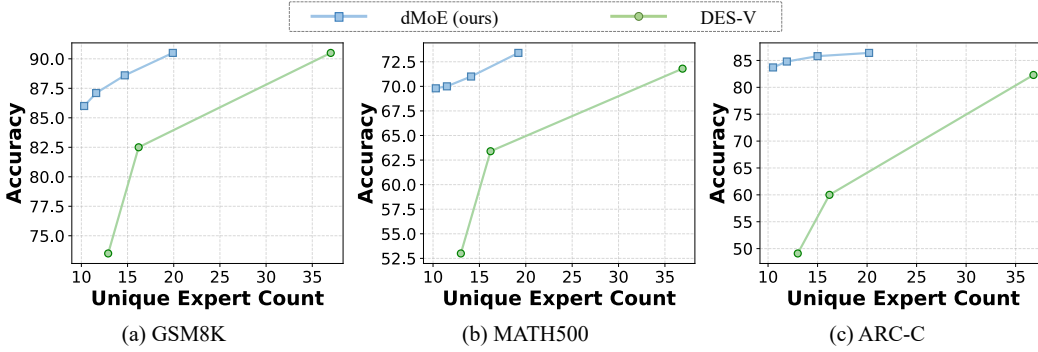


Figure 6: Comparison of the performance-efficiency trade-off between our method and baselines. We report the results of GSM8K (a), MATH500 (b), and ARC-C (c).

**Ablation on the Block Size.** We further conduct an ablation study on the block size, with the results summarized in Table 4. Specifically, we evaluate the model under block sizes of 8, 16, 24, and 32. The results show that our dMoE consistently compresses the unique expert count while maintaining strong performance across different block sizes. Specifically, under block sizes of 8, 16, 24, and 32, our dMoE reduces the average unique expert count by 79.04%, 73.82%, 66.33%, and 53.92%, while retaining 99.11%, 100.63%, 99.53%, and 99.64% of the original performance, respectively. The consistent efficiency improvement without a performance drop across different block sizes further demonstrates the effectiveness of our method.

**dMoE Reaches Superior Performance-Efficiency Trade-off.** We present the performance-efficiency trade-off between our dMoE and the baselines. The results of our dMoE are taken from Table 3. For DES, we compare against its stronger variant, DES-V, and obtain its trade-off points by varying the threshold. As shown in Figure 6, our dMoE achieves better performance with fewer unique experts, leading to a superior performance-efficiency trade-off.

## 6 Conclusion

In this work, we introduce dMoE, a novel strategy for block-level expert routing in MoE dLLMs. dMoE enables aggressive expert compression while incurring almost no performance degradation. Comprehensive evaluations across diverse benchmarks demonstrate the effectiveness of our method. More broadly, this work lays a foundation for block-level routing in MoE dLLMs and opens up a promising direction for improving their efficiency.

## References

- [1] Qiuhua Yi, Xiangfan Chen, Chenwei Zhang, Zehai Zhou, Linan Zhu, and Xiangjie Kong. Diffusion models in text generation: a survey. *PeerJ Computer Science*, 2024.
- [2] Lingzhe Zhang, Liancheng Fang, Chiming Duan, Minghua He, Leyi Pan, Pei Xiao, Shiyu Huang, Yunpeng Zhai, Xuming Hu, Philip S Yu, et al. A survey on parallel text generation: From parallel decoding to diffusion language models. *arXiv preprint arXiv:2508.08712*, 2025.
- [3] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- [4] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- [5] Runpeng Yu, Xinyin Ma, and Xinchao Wang. Dimple: Discrete diffusion multimodal large language model with parallel decoding. *arXiv preprint arXiv:2505.16990*, 2025.
- [6] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [7] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [8] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [9] Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuehang Yang, Hongli Yu, Xingwei Qu, et al. Seed diffusion: A large-scale diffusion language model with high-speed inference. *arXiv preprint arXiv:2508.02193*, 2025.
- [10] Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, Stefano Ermon, et al. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.
- [11] Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, et al. Llada2. 0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025.
- [12] Tiwei Bie, Maosong Cao, Xiang Cao, Bingsen Chen, Fuyuan Chen, Kun Chen, Lun Du, Daozhuo Feng, Haibo Feng, Mingliang Gong, Zhuocheng Gong, Yanmei Gu, Jian Guan, Kaiyuan Guan, Hongliang He, Zenan Huang, Juyong Jiang, Zhonghui Jiang, Zhenzhong Lan, Chengxi Li, Jianguo Li, Zehuan Li, Huabin Liu, Lin Liu, Guoshan Lu, Yuan Lu, Yuxin Ma, Xingyu Mou, Zhenxuan Pan, Kaida Qiu, Yuji Ren, Jianfeng Tan, Yiding Tian, Zian Wang, Lanning Wei, Tao Wu, Yipeng Xing, Wentao Ye, Liangyu Zha, Tianze Zhang, Xiaolu Zhang, Junbo Zhao, Da Zheng, Hao Zhong, Wanli Zhong, Jun Zhou, Junlin Zhou, Liwang Zhu, Muzhi Zhu, and Yihong Zhuang. Llada2.1: Speeding up text diffusion via token editing, 2026.
- [13] Fengqi Zhu, Zebin You, Yipeng Xing, Zenan Huang, Lin Liu, Yihong Zhuang, Guoshan Lu, Kangyu Wang, Xudong Wang, Lanning Wei, Hongrui Guo, Jiaqi Hu, Wentao Ye, Tieyuan Chen, Chenchen Li, Chengfu Tang, Haibo Feng, Jun Hu, Jun Zhou, Xiaolu Zhang, Zhenzhong Lan, Junbo Zhao, Da Zheng, Chongxuan Li, Jianguo Li, and Ji-Rong Wen. Llada-moe: A sparse moe diffusion language model. *arXiv preprint arXiv:2509.24389*, 2025.
- [14] Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, et al. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.
- [15] Jinjie Ni and team. Openmoe 2: Sparse diffusion language models. <https://github.com/JinjieNi/OpenMoE2>, 2025.
- [16] Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [17] Enshu Liu, Junyi Zhu, Zinan Lin, Xuefei Ning, Matthew B Blaschko, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. Efficient expert pruning for sparse mixture-of-experts language models: Enhancing performance and reducing inference costs. *arXiv preprint arXiv:2407.00945*, 2024.

- [18] Tianyu Chen, Shaohan Huang, Yuan Xie, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. Task-specific expert pruning for sparse mixture-of-experts. *arXiv preprint arXiv:2206.00277*, 2022.
- [19] Mohammed Nowaz Rabbani Chowdhury, Meng Wang, Kaoutar El Maghraoui, Naigang Wang, Pin-Yu Chen, and Christopher Carothers. A provably effective method for pruning experts in fine-tuned sparse mixture-of-experts. *arXiv preprint arXiv:2405.16646*, 2024.
- [20] Hongcheng Guo, Juntao Yao, Boyang Wang, Junjia Du, Shaosheng Cao, Donglin Di, Shun Zhang, and Zhoujun Li. Cluster-driven expert pruning for mixture-of-experts large language models. *arXiv preprint arXiv:2504.07807*, 2025.
- [21] Chenyang Song, Weilin Zhao, Xu Han, Chaojun Xiao, Yingfa Chen, Yuxuan Li, Zhiyuan Liu, and Maosong Sun. Blockffn: Towards end-side acceleration-friendly mixture-of-experts with chunk-level activation sparsity. *arXiv preprint arXiv:2507.08771*, 2025.
- [22] Shwai He, Run-Ze Fan, Liang Ding, Li Shen, Tianyi Zhou, and Dacheng Tao. Merging experts into one: Improving computational efficiency of mixture of experts. In *EMNLP*, 2023.
- [23] Sejik Park. Learning more generalized experts by merging experts in mixture-of-experts. *arXiv preprint arXiv:2405.11530*, 2024.
- [24] Lujun Li, Qiyuan Zhu, Jiacheng Wang, Xiaoyu Qin, Wei Li, Hao Gu, Sirui Han, and Yike Guo. Sub-moe: Efficient mixture-of-expert llms compression via subspace expert merging. In *AAAI*, 2026.
- [25] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. In *ACL*, 2024.
- [26] Yushi Huang, Zining Wang, Zhihang Yuan, Yifu Ding, Ruihao Gong, Jinyang Guo, Xianglong Liu, and Jun Zhang. Modes: Accelerating mixture-of-experts multimodal large language models via dynamic expert skipping. *arXiv preprint arXiv:2511.15690*, 2025.
- [27] Maryam Akhavan Aghdam, Hongpeng Jin, and Yanzhao Wu. Da-moe: Towards dynamic expert allocation for mixture-of-experts models. *arXiv preprint arXiv:2409.06669*, 2024.
- [28] Yuanteng Chen, Yuantian Shao, Peisong Wang, and Jian Cheng. Eac-moe: Expert-selection aware compressor for mixture-of-experts large language models. In *ACL*, 2025.
- [29] Zheyue Tan, Zhiyuan Li, Tao Yuan, Dong Zhou, Weilin Liu, Yueqing Zhuang, Yadong Li, Guowei Niu, Cheng Qin, Zhu Yu Yao, et al. Rexmoe: Reusing experts with minimal overhead in mixture-of-experts. *arXiv preprint arXiv:2510.17483*, 2025.
- [30] Costin-Andrei Oncescu, Qingyang Wu, Wai Tong Chung, Robert Wu, Bryan Gopal, Junxiong Wang, Tri Dao, and Ben Athiwaratkun. Opportunistic expert activation: Batch-aware expert routing for faster decode without retraining. *arXiv preprint arXiv:2511.02237*, 2025.
- [31] Shuibai Zhang, Caspian Zhuang, Chihan Cui, Zhihan Yang, Fred Zhangzhi Peng, Yanxin Zhang, Haoyue Bai, Zack Jia, Yang Zhou, Guanhua Chen, et al. Expert-choice routing enables adaptive computation in diffusion language models. *arXiv preprint arXiv:2604.01622*, 2026.
- [32] Linye Wei, Zixiang Luo, Pingzhi Tang, and Meng Li. Team: Temporal-spatial consistency guided expert activation for moe diffusion language model acceleration. *arXiv preprint arXiv:2602.08404*, 2026.
- [33] Hao Mark Chen, Zhiwen Mo, Royson Lee, Qianzhou Wang, Da Li, Shell Xu Hu, Wayne Luk, Timothy Hospedales, and Hongxiang Fan. Dynamic expert sharing: Decoupling memory from parallelism in mixture-of-experts diffusion llms. *arXiv preprint arXiv:2602.00879*, 2026.
- [34] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *ICLR*, 2023.
- [35] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [36] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

- [37] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *ICLR*, 2021.
- [38] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.
- [39] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *ICCV*, 2023.
- [40] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. In *NeurIPS*, 2022.
- [41] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, et al. Video generation models as world simulators. *OpenAI Blog*, 1(8):1, 2024.
- [42] Haohe Liu, Zehua Chen, Yi Yuan, Xinhao Mei, Xubo Liu, Danilo Mandic, Wenwu Wang, and Mark D Plumbley. Audioldm: Text-to-audio generation with latent diffusion models. *arXiv preprint arXiv:2301.12503*, 2023.
- [43] Zach Evans, CJ Carr, Josiah Taylor, Scott H Hawley, and Jordi Pons. Fast timing-conditioned latent audio diffusion. In *ICML*, 2024.
- [44] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- [45] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *NeurIPS*, 2019.
- [46] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.
- [47] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. In *NeurIPS*, 2021.
- [48] Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *NeurIPS*, 2024.
- [49] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.
- [50] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. *arXiv preprint arXiv:2409.02908*, 2024.
- [51] Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, et al. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.
- [52] Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *arXiv preprint arXiv:2504.12216*, 2025.
- [53] Xiaohang Tang, Rares Dolga, Sangwoong Yoon, and Ilija Bogunovic. wd1: Weighted policy optimization for reasoning in diffusion language models. *arXiv preprint arXiv:2507.08838*, 2025.
- [54] Nianyi Lin, Jiajie Zhang, Lei Hou, and Juanzi Li. Boundary-guided policy optimization for memory-efficient rl of diffusion large language models. *arXiv preprint arXiv:2510.11683*, 2025.
- [55] Sicheng Feng, Zigeng Chen, Xinyin Ma, Gongfan Fang, and Xinchao Wang. dvoting: Fast voting for dllms. *arXiv preprint arXiv:2602.12153*, 2026.
- [56] Sicheng Feng, Gongfan Fang, Xinyin Ma, and Xinchao Wang. Efficient reasoning models: A survey. *arXiv preprint arXiv:2504.10903*, 2025.
- [57] Ling Yang, Ye Tian, Bowen Li, Xinchao Zhang, Ke Shen, Yunhai Tong, and Mengdi Wang. Mmada: Multimodal large diffusion language models. *arXiv preprint arXiv:2505.15809*, 2025.
- [58] Shufan Li, Konstantinos Kallidromitis, Hritik Bansal, Akash Gokul, Yusuke Kato, Kazuki Kozuka, Jason Kuen, Zhe Lin, Kai-Wei Chang, and Aditya Grover. Lavida: A large diffusion language model for multimodal understanding. *arXiv preprint arXiv:2505.16839*, 2025.

- [59] Zebin You, Shen Nie, Xiaolu Zhang, Jun Hu, Jun Zhou, Zhiwu Lu, Ji-Rong Wen, and Chongxuan Li. Llada-v: Large language diffusion models with visual instruction tuning. *arXiv preprint arXiv:2505.16933*, 2025.
- [60] Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. Diffucoder: Understanding and improving masked diffusion models for code generation. *arXiv preprint arXiv:2506.20639*, 2025.
- [61] Fred Zhangzhi Peng, Shuibai Zhang, and Alex Tong. contributors. open-dllm: Open diffusion large language models.
- [62] Runpeng Yu, Qi Li, and Xinchao Wang. Discrete diffusion in large language and multimodal models: A survey. *arXiv preprint arXiv:2506.13759*, 2025.
- [63] Tianyi Li, Mingda Chen, Bowei Guo, and Zhiqiang Shen. A survey on diffusion language models. *arXiv preprint arXiv:2508.10875*, 2025.
- [64] Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. Dmax: Aggressive parallel decoding for dllms. *arXiv preprint arXiv:2604.08302*, 2026.
- [65] Rishov Sarkar, Hanxue Liang, Zhiwen Fan, Zhangyang Wang, and Cong Hao. Edge-moe: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts. In *ICCAD*, 2023.
- [66] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2022.
- [67] Linfeng Zhang, Chenglong Bao, and Kaisheng Ma. Self-distillation: Towards efficient and compact neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(8):4388–4403, 2021.
- [68] Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13(9):9, 2024.
- [69] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, et al. Openthoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178*, 2025.
- [70] Sicheng Feng, Song Wang, Shuyi Ouyang, Lingdong Kong, Zikai Song, Jianke Zhu, Huan Wang, and Xinchao Wang. Can mllms guide me home? a benchmark study on fine-grained visual reasoning from transit maps. *arXiv preprint arXiv:2505.18675*, 2025.
- [71] Kele Shao, Keda Tao, Kejia Zhang, Sicheng Feng, Mu Cai, Yuzhang Shang, Haoxuan You, Can Qin, Yang Sui, and Huan Wang. When tokens talk too much: A survey of multimodal long-context token compression across images, videos, and audios. *arXiv preprint arXiv:2507.20198*, 2025.
- [72] Junhan Zhu, Hesong Wang, Mingluo Su, Zefang Wang, and Huan Wang. Obs-diff: Accurate pruning for diffusion models in one-shot. *arXiv preprint arXiv:2510.06751*, 2025.
- [73] Sicheng Feng, Kaiwen Tuo, Song Wang, Lingdong Kong, Jianke Zhu, and Huan Wang. Rewardmap: Tackling sparse rewards in fine-grained visual reasoning via multi-stage reinforcement learning. *arXiv preprint arXiv:2510.02240*, 2025.
- [74] Keda Tao, Kele Shao, Bohan Yu, Weiqiang Wang, Huan Wang, et al. Omnizip: Audio-guided dynamic token compression for fast omnimodal large language models. *arXiv preprint arXiv:2511.14582*, 2025.
- [75] Xin Jin, Siyuan Li, Siyong Jian, Kai Yu, and Huan Wang. Mergemix: A unified augmentation paradigm for visual and multi-modal understanding. *arXiv preprint arXiv:2510.23479*, 2025.
- [76] Zhenxin Ai and Haiyun He. Pasa: A principled embedding-space watermarking approach for llm-generated text under semantic-invariant attacks. *arXiv preprint arXiv:2605.10977*, 2026.
- [77] Wenjie Du, Li Jiang, Keda Tao, Xue Liu, and Huan Wang. Which heads matter for reasoning? rl-guided kv cache compression. In *ICML*, 2026.

## Appendix

In Appendix A, we discuss limitations and future work. We further provide the impact statement in Appendix B and license statement in Appendix C, and computing resources in Appendix D.

|                                        |           |
|----------------------------------------|-----------|
| <b>A Limitations &amp; Future Work</b> | <b>14</b> |
| <b>B Impact Statement</b>              | <b>14</b> |
| <b>C License Statement</b>             | <b>14</b> |
| <b>D Computing Resources</b>           | <b>15</b> |

### A Limitations & Future Work

Our method is, in principle, applicable to any MoE dLLMs. In this work, we focus our evaluation on the language modality, but the same idea can be naturally extended to other modalities, such as image and video, and to downstream tasks including visual question answering [70, 71, 72] and visual reasoning [73, 74, 75, 76, 77]. More broadly, our results suggest that block-level expert routing is a promising direction for future MoE dLLMs. Looking ahead, this direction can be pushed further toward more extreme compression, for example, by encouraging all tokens within a block to share the same expert group or by jointly reducing computation cost through selecting fewer experts for each token.

### B Impact Statement

This work studies efficient inference for MoE dLLMs by reducing the number of uniquely activated experts during decoding. By implementing block-level expert routing, the proposed method reduces memory overhead, lowers inference cost, and alleviates the memory bottleneck in MoE dLLMs. These improvements make large-scale model deployment more practical, especially in resource-constrained, memory-sensitive, or latency-sensitive settings. More broadly, this work contributes to improving the efficiency and accessibility of dLLMs in real-world applications. Furthermore, the method does not introduce additional ethical or societal concerns.

### C License Statement

The model and datasets used in this paper are publicly available, and all experiments are conducted in compliance with their respective licenses. The specific licenses for the model and datasets are listed below.

- LLaDA2.0-mini<sup>5</sup> ..... Apache 2.0 License
- GSM8K<sup>6</sup> ..... MIT License
- MATH500<sup>7</sup> ..... MIT License
- ARC-C<sup>8</sup> ..... cc-by-sa-4.0 License
- MMLU<sup>9</sup> ..... MIT License

To ensure fair and reproducible evaluation, we follow the official documentation and recommended practices of each model when implementing the inference procedures.

<sup>5</sup><https://huggingface.co/inclusionAI/LLaDA2.0-mini>.

<sup>6</sup><https://huggingface.co/datasets/openai/gsm8k>.

<sup>7</sup><https://huggingface.co/datasets/HuggingFaceH4/MATH-500>

<sup>8</sup>[https://huggingface.co/datasets/allenai/ai2\\_arc](https://huggingface.co/datasets/allenai/ai2_arc)

<sup>9</sup><https://huggingface.co/datasets/cais/mmlu>

## **D Computing Resources**

The experiments were performed on a server equipped with two AMD EPYC 9654 96-Core processors, 1.5 TiB of system memory, and four NVIDIA H100 GPUs with 80 GB VRAM each. We employed data parallelism to speed up evaluation. Each evaluation run was completed within 12 hours, while each training run finished within 144 hours.